

Plano de Ação — Nova Etapa PLAYBOOK na Esteira

Fábrica Agêntica de Publicações · análise por grafo + esforço + implementação

Fábrica Agêntica de Publicações

Agosto 2026

Sumário

1. Sumário executivo	2
2. Análise da estrutura via grafo	2
2.1 Comunidades detectadas	2
2.2 Os 6 pontos de dispatch por tipo_obra (o “eixo de extensão”)	3
2.3 O precedente que barateia tudo: o e-book	3
2.4 A alavanca determinística	3
3. O que o PLAYBOOK deve ser (requisito do operador)	4
3.1 Estrutura de saída proposta	4
4. Esforço detalhado	5
4.1 Arquivos NOVOS (5)	5
4.2 Arquivos ALTERADOS (11) — todas as mudanças aditivas	6
4.3 Testes (rede de proteção)	8
4.4 Consolidado do esforço	8
5. Plano de ação — fase a fase	8
Fase A — Contrato (2 h)	8
Fase B — Extração determinística (4 h) — <i>o coração da etapa</i>	9
Fase C — Identidade visual (1 h)	10
Fase D — Compilação (3 h)	10
Fase E — Gate (3 h)	11
Fase F — Orquestração (1,5 h)	11
6. Ordem de execução e dependências	12
7. Caminho enxuto (10 h) — se o objetivo for validar rápido	12
8. Riscos e mitigações	13
9. Impacto em tokens	14
10. Definição de pronto (aceite final da etapa)	14
11. Prova de conceito já disponível	14

1. Sumário executivo

Item	Resultado
Pergunta	Qual o esforço para adicionar uma etapa de PLAYBOOK após a geração do livro?
Veredito	Baixo–médio. 5 arquivos novos + 11 alterações cirúrgicas, todas aditivas
Volume	~470 linhas novas · ~65 linhas alteradas · 0 linha removida
Esforço	10–14 h de trabalho assistido (2 sessões) ou 3 ciclos de fábrica
Risco	Baixo — nenhuma alteração destrutiva; retro-compatibilidade por <code>setDefault</code>
Alavanca principal	O framework EITA-V2 tem 7 seções fixas ; extrair §4 <i>Técnica</i> + §5 <i>Aplica</i> é parse determinístico , não geração por LLM
Economia	~70% do playbook sai de script determinístico (0 token); LLM só escreve capa, objetivo e polimento dos cards

2. Análise da estrutura via grafo

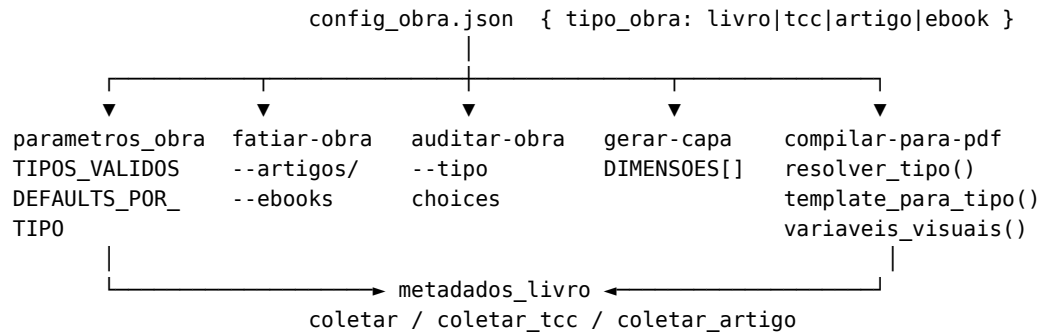
Fonte: `.code-review-graph/graph.db` (schema v9, 401 nós, 4.305 arestas, 45 fluxos, 7 comunidades).
Consulta somente-leitura — **nenhum arquivo de código foi lido por inteiro para o mapeamento**; apenas 6 leituras cirúrgicas nos pontos de extensão confirmados pelo grafo.

2.1 Comunidades detectadas

Comunidade	Tamanho	Linguagem	Papel na esteira
<code>scripts-validar</code>	168 nós	Python	Núcleo determinístico — auditoria, validação, fatiamento, capas
<code>tests-regex</code>	82 nós	Python	Rede de proteção (4 arquivos de teste)
<code>scripts-code</code>	27 nós	Python	Token economy (cave-man)
<code>pdf-gen-server-gerar</code>	24 nós	JavaScript	Fallback CloudConvert
<code>legado-capitulo</code>	25 nós	Python	<code>scripts/_legado/</code> — não tocar
<code>compilar-para-pdf</code>	11 nós	Python	Compilação (dispatch por <code>tipo_obra</code>)
<code>compilar-mega-livro</code>	5 nós	Python	Consolidação multi-obra

Leitura: a fábrica é um **monólito de scripts determinísticos** orquestrado por comandos em Markdown. Não há framework, injeção de dependência ou registro de plugins — a extensão se faz por **dispatch em tipo_obra**, replicado em 6 pontos.

2.2 Os 6 pontos de dispatch por tipo_obra (o “eixo de extensão”)



Consequência prática: adicionar playbook é o mesmo movimento que a V4 já fez ao adicionar artigo e ebook. O caminho está trilhado — inclusive com um precedente quase idêntico (ebook), que também é **derivado do livro-mãe e não pesquisa nada**.

2.3 O precedente que barateia tudo: o e-book

Aspecto	E-book (existente)	Playbook (proposto)
Origem	Capítulos do livro-mãe	Capítulos do livro-mãe
Pesquisa	Nunca (RAG/dossiê já pronto)	Nunca
Fatiamento	fatiar-obra.py --ebooks	fatiar-obra.py --playbook
Transformação	Reescrita de tom por LLM (custosa)	Extração de seções por parser (barata)
Capa	gerar-capla.py --tipo ebook + ilustração condutora	idem, --tipo playbook
Saída	.epub + .pdf	.md + .pdf

O playbook é **mais barato que o e-book**: onde o e-book precisa de um subagente reescrevendo prosa, o playbook precisa de um re.split nos cabeçalhos EITA.

2.4 A alavanca determinística

auditar-obra.py já possui as funções exatas de que o extrator precisa:

Função existente	Linhas	Reuso no playbook
cabecalho_secao(numero, nome)	108–111	Localizar ## 4. Técnica e ## 5. Aplica
dividir_secoes(texto)	114–123	Particionar o capítulo nas 7 seções EITA
auditar_capitulo(caminho, vocabulario)	198–295	Já valida que as 7 seções existem

E sumario_macro.json já carrega o **motivo condutor** estruturado — exatamente o que o requisito “seguir a ilustração condutora do livro” pede:

```

"motivo_condutor": {
  "nome": "A Obra em Construção",
  "descricao": "O leitor é o Mestre de Obras de um projeto real...",
  "vocabulario": ["canteiro de obras", "fundação", "andares", "estrutura",
                  "instalações", "acabamento", "entrega das chaves",
                  "mestre de obras", "plantas", "inspeção"],
  "persona_leitor": "Mestre de Obras"
}

```

Nada precisa ser inventado: o vocabulário condutor vira o nome dos estágios do playbook, a persona vira o “você” do texto, e a descrição vira o briefing da ilustração de capa (subagente-ilustrador, Modo Capa, já existente).

3. O que o PLAYBOOK deve ser (requisito do operador)

#	Requisito	Fonte determinística	Quem produz
P1	CAPA com badge de nível	config_obra.json.senioridade + sumario_macro.json	compilar-capas.py --tipo playbook
P2	Ilustração da capa do motivo condutor	motivo_condutor.descricao	subagente-ilustrador (Modo Capa)
P3	OBJETIVO DO MATERIAL	motivo_condutor + introducao do sumário	LLM (1 parágrafo, ~15 linhas)
P4	Seguir a ilustração condutora do livro	motivo_condutor.vocabulario + persona_leitor	Extrator (nomes de estágio) + LLM (ligações)
P5	PASSOS PRÁTICOS do início ao fim	§4 Técnica + §5 Aplicação de cada cap_NN.md	Extrator determinístico
P6	PDF no padrão da editora	template_playbook.typ	compilar-para-pdf.py --tipo playbook

3.1 Estrutura de saída proposta

```

PLAYBOOK-<slug-curto>.md / .pdf
├─ Capa gráfica (badge de nível + ilustração condutora)      ← P1, P2
├─ Objetivo do material (1 página)                            ← P3
├─ Como usar + persona + vocabulário do motivo condutor     ← P4
├─ Mapa dos Estágios (1 estágio por Parte do livro)        ← P4
├─ PASSOS PRÁTICOS – 1 card por capítulo, na ordem do livro ← P5
│   └─ ① Objetivo do passo (do `objetivo` do sumário)
│   └─ ② Pré-requisito (card anterior)
│   └─ ③ Entregas (arquivos citados na §4 Técnica)
│   └─ ④ Execução (blocos e comandos da §4 Técnica)
│   └─ ⑤ Verificação / Gate (script citado na §4)
│   └─ ⑥ Feito quando... (do Exercício Prático da §5)
│   └─ ⑦ Armadilhas (das "Armadilhas Comuns" da §5)
└─ Fechamento + checklist mestre (1 folha)                  ← P5

```

Regra de ouro do playbook (proposta como R-PBK-0): o playbook **não repete teoria**. As seções §1 *Introdução*, §2 *Explica*, §3 *Ilustra* e §7 *Referências* do livro **não entram** — apenas viram referência cruzada → Cap. N.

4. Esforço detalhado

4.1 Arquivos NOVOS (5)

#	Arquivo	Linhas	Molde existente	Esforço
N1	SPEC_PLAYBOOK.md	~150	SPEC_EBOOK.md	1 h
N2	scripts/ extrair- passos- praticos.py	~180	reusa dividir_secoes de auditar- obra.py	3 h
N3	scripts/ validar- playbook.py	~120	validar-cap- nivel.py + auditar- obra.py	2 h
N4	templates/template_playbook.typ	~90	template.typ (fork enxuto)	2 h
N5	.claude/ commands/ criar- playbook.md	~60	.claude/ commands/ criar-ebook.md	0,5 h

Opcional (não bloqueia a v1): .claude/agents/subagente-redator-playbook.md (~80 linhas) para polimento em lote quando o livro tiver mais de 12 capítulos.

4.2 Arquivos ALTERADOS (11) – todas as mudanças aditivas

#	Arquivo	Ponto exato	Δ linhas
A1	scripts/ parametros_obra.py	TIP0S_VALID0S (L31) · DEFAULTS_P0R_TIP0 (L49) · carregar_config setdefaults (L114-117) · validar_config (L146)	+10
A2	scripts/fatiar- obra.py	nova gerar_playbook(slug) (molde de gerar_ebooks, L177) · flag --playbook no main (L253)	+55
A3	compilar-para- pdf.py	TEMPLATE_PLAYBOOK (L97) · template_para_tipo (L116) · ramo em variaveis_visuais (L238) · --number- sections só para livro (L290)	+14
A4	scripts/gerar- capa.py	DIMENS0ES["playbook"] · --tipo choices (L220) · gerar_capa_da_obra aceita playbook (L186)	+5
A5	scripts/ metadados_livro.py	coletar_playbook() variaveis_pandoc_playbook() (molde de coletar_artigo, L310)	+45
A6	scripts/auditar- obra.py	--tipo choices += playbook (L635); de- lega o mérito a validar-playbook.py	+2
A7	scripts/empacotar- distribuicao.py	copiar PLAYBOOK- *.pdf + li- nha no README (montar_readme, L51)	+18
A8	.claude/commands/ esbocar.md	+1 pergunta na Ro- dada 2 · campo gerar_playbook no schema	+6
A9	.claude/commands/ produzir-obra- completa.md	3º ramo no Passo 2 (paralelo a artigos/e- -books)	+8
A10	.claude/commands/ criar-livro.md	novo Passo 4.5 (opcio- nal, quando roda sozi- tabela "Tipos de Obra")	+7
A11	CLAUDE.md	· lista de scripts · squad	+6

CLAUDE.md é hardlink de AGENTS.md, .cursor/rules/, .windsurfrules, .clinerules e .github/copilot-instructions.md — **uma edição propaga para os 6**. Custo zero.

4.3 Testes (rede de proteção)

Arquivo	Casos	Esforço
tests/ test_extrair_passos.py	capítulo EITA completo · capítulo sem §4 · capítulo truncado · acentuação	1,5 h
tests/ test_validar_playbook.py	R-PBK-1 a R-PBK-5 (positivo e negativo de cada)	1 h

O projeto já roda pytest com 4 suítes (82 nós no grafo) — os novos testes seguem o mesmo padrão de test_auditar_obra.py.

4.4 Consolidado do esforço

Fase	Escopo	Horas
A — Contrato	N1 (SPEC_PLAYBOOK.md) + A1 (parametros_obra) + A8 (esbocar)	2,0
B — Extração	N2 (extrair-passos-praticos.py) + A2 (fatiar-obra)	4,0
C — Identidade visual	A4 (gerar-capa) + reuso do subagente-ilustrador	1,0
D — Compilação	N4 (template_playbook.typ) + A3 + A5	3,0
E — Gate	N3 (validar-playbook.py) + A6 + testes	3,0
F — Orquestração	N5 (criar-playbook.md) + A7 + A9 + A10 + A11	1,5
TOTAL		14,5 h (10 h no caminho enxuto, ver §7)

5. Plano de ação — fase a fase

Fase A — Contrato (2 h)

A.1 Criar SPEC_PLAYBOOK.md na raiz, espelhando SPEC_EBOOK.md, com as regras:

Regra	Enunciado
R-PBK-0	Playbook não repete teoria — §1, §2, §3 e §7 do livro nunca entram
R-PBK-1	Todo card tem as 7 partes (① a ⑦)
R-PBK-2	Todo card cita ao menos 1 entrega com caminho de arquivo
R-PBK-3	Todo card tem 1 comando de verificação executável
R-PBK-4	“Feito quando...” tem de 3 a 7 itens binários
R-PBK-5	Nenhuma parte do card excede 25 linhas (documento de bancada)
R-PBK-6	Capa com badge de nível + ilustração no motivo condutor (herda REGRA 5)
R-PBK-7	1 card por capítulo do livro-mãe, na mesma ordem , sem lacuna
R-PBK-8	Vocabulário do motivo_condutor presente nos nomes de estágio

A.2 scripts/parametros_obra.py:

```
TIPOS_VALIDOS = ("livro", "tcc", "artigo", "ebook", "playbook") # L31
DEFAULTS_POR_TIPO["playbook"] = {"min_refs": 0} # L49
# carregar_config (L114-117):
dados.setdefault("gerar_playbook", False)
# validar_config: playbook não tem qtd (é sempre 1 por livro) → sem faixa a validar
```

A.3 .claude/commands/esbocar.md — Rodada 2 ganha:

Header	Pergunta	Condição	Opções
Playbook	Deseja gerar o Playbook prático da obra?	Tipo = Livro	Sim (Recommended) Não

E o schema de config_obra.json ganha "gerar_playbook": true.

Critério de aceite A: python scripts/parametros_obra.py <slug> --validar aceita um config com gerar_playbook e continua aceitando configs antigos sem o campo.

Fase B — Extração determinística (4 h) — o coração da etapa

B.1 scripts/extrair-passos-praticos.py:

Entrada : output/<slug>/capitulos/cap_NN.md (+ sumario_macro.json)
Saída : output/playbooks/<slug>--pbk/passos/passo_NN.json

Para cada capítulo:

1. dividir_secoes(texto) # reuso de auditar-obra.py
2. tecnica = secoes["4. Técnica"]
aplica = secoes["5. Aplica"]
3. entregas = caminhos de arquivo citados em `tecnica` (regex de path/backtick)
4. execucao = subtítulos "### ..." + blocos de código de `tecnica`
5. gate = 1ª linha de comando executável (python|pytest|bash|git|node)
6. feito = "### Exercício Prático" de `aplica` → itens binários

```
7. armadilhas = "### Armadilhas Comuns..." de `aplica` → 3 primeiros itens
8. objetivo = sumario_macro.partes[].capitulos[].objetivo
```

Modo `--relatorio` imprime, por capítulo, o que **não** foi encontrado (capítulo sem §4, sem exercício, sem comando) — vira a lista de trabalho do polimento por LLM.

B.2 `scripts/fatiar-obra.py` — nova `gerar_playbook(slug)` (molde de `gerar_ebooks`, L177-247), que cria:

```
output/playbooks/<slug_mae>--pbk/
├─ config_obra.json      { tipo_obra: "playbook", livro_mae: <slug>, senioridade
herdada }
├─ sumario_macro.json    { titulo, motivo_condutor herdado, estagios[] = partes[] }
├─ passos/               (preenchido por extrair-passos-praticos.py)
├─ imagens/
└─ revisao/
```

e grava a seção `playbook` em `derivados.json` do livro-mãe (preservando artigos e ebooks, exatamente como `gerar_ebooks` já faz).

`main()` ganha `--playbook` no grupo mutuamente exclusivo.

Critério de aceite B: rodar sobre o livro AIDD v2 (20 capítulos) produz 20 `passo_NN.json`, todos com objetivo, execucao e gate não vazios; o relatório aponta zero capítulo sem §4.

Fase C — Identidade visual (1 h)

C.1 `scripts/gerar-capa.py`: `DIMENSÕES["playbook"] = DIMENSÕES["livro"]` (A4), `--tipo` aceita `playbook`, e `gerar_capa_da_obra` reconhece o prefixo `playbooks/`.

C.2 Reuso do subagente-ilustrador (Modo Capa) — **sem alteração no agente**. O briefing passa a ser `motivo_condutor.descricao` do livro-mãe, garantindo que a ilustração do `playbook` seja o **mesmo canteiro de obras** do livro. Saída em `output/playbooks/<slug>--pbk/imagens/capa_ilustracao.png`.

C.3 O badge de nível continua obrigatório e validado por `validar-capa-nivel.py` (já genérico: recebe `dir_obra`, não depende do tipo). **Zero alteração**.

Critério de aceite C: `validar-capa-nivel.py output/playbooks/<slug>--pbk` retorna 0 e a capa exhibe a mesma paleta (`series_capa.resolver_cor`) do livro-mãe.

Fase D — Compilação (3 h)

D.1 `templates/template_playbook.typ` — fork enxuto de `template.typ`: mantém capa gráfica e sumário; **remove** ficha catalográfica ABNT, folha de rosto e seção de referências; **adiciona** o bloco “Objetivo do Material” e o estilo de card (caixa com filete na cor de acento).

D.2 `compilar-para-pdf.py`:

```
TEMPLATE_PLAYBOOK = DIR_PROJETO / "templates" / "template_playbook.typ" # L97
```

```
def template_para_tipo(tipo): # L116
    if tipo == "playbook" and TEMPLATE_PLAYBOOK.exists():
        return TEMPLATE_PLAYBOOK
    ...
```

```
# variaveis_visuais (L238): ramo playbook → metadados_livro.coletar_playbook()
```

```
# comando_pandoc (L290): --number-sections apenas para tipo == "livro"
```

D.3 `scripts/metadados_livro.py` — `coletar_playbook()` (molde de `coletar_artigo`, L310-338) devolve: título, subtítulo, objetivo, livro-mãe, nº de passos, persona, cor de acento, caminho da capa. Sem ISBN/CIP/CDD (`playbook` não é obra catalogada).

Critério de aceite D: `python compilar-para-pdf.py playbooks/<slug>--pbk --tipo playbook` gera PDF > 0 bytes com capa gráfica, sem seção de referências e sem CIP.

Fase E — Gate (3 h)

E.1 `scripts/validar-playbook.py` implementa R-PBK-0 a R-PBK-8, com `--estrito` (exit 1) e `--json`, no mesmo contrato de saída de `auditar-obra.py`.

E.2 `scripts/auditar-obra.py`: `--tipo aceita` `playbook` e, nesse caso, **delega** para `validar-playbook.py` em vez de aplicar requisitos de livro (que exigiriam referências e 7 seções EITA — inaplicáveis).

E.3 Testes `tests/test_extrair_passos.py` e `tests/test_validar_playbook.py`.

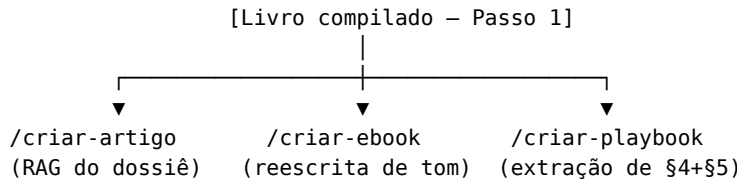
Critério de aceite E: `pytest tests/ -q verde`; um `playbook` com card faltando a parte ⑤ é reprovado por R-PBK-3 com mensagem apontando o número do card.

Fase F — Orquestração (1,5 h)

F.1 `.claude/commands/criar-playbook.md` (molde de `criar-ebook.md`):

Pré-condição : `output/<slug>/capitulos/cap_*.md` existem (Fase 2 do livro rodou)
Passo 1 : `fatiar-obra.py <slug> --playbook`
Passo 2 : `extrair-passos-praticos.py <slug>` (determinístico, 0 token)
Passo 3 : polimento por LLM apenas do que o relatório apontou como lacuna
+ Objetivo do Material + ligações entre estágios
Passo 4 : `subagente-ilustrador (Modo Capa) + gerar-capa.py --tipo playbook`
Passo 5 : `validar-playbook.py --estrito` (até 2 rodadas de correção)
Passo 6 : `compilar-para-pdf.py playbooks/<slug>--pbk --tipo playbook`
Passo 7 : relatório telegráfico (REGRA 2)

F.2 `produzir-obra-completa.md` — Passo 2 vira três ramos paralelos:



F.3 `criar-livro.md` — Passo 4.5: se `gerar_playbook=true` e o comando roda sozinho, dispara `/criar-playbook <slug>` antes da distribuição.

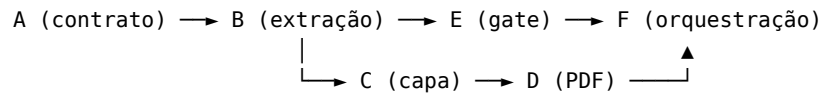
F.4 `empacotar-distribuicao.py` — copia `PLAYBOOK-<slug>.pdf` para `distribuicao/` e acrescenta a linha correspondente no README gerado.

F.5 `CLAUDE.md` — tabela de Tipos de Obra ganha a linha:

Tipo	Spec	Comando	Redator	Compilador
Playbook	SPEC_PLAYBOOK.md	/ criar-playbook	extrair-passos-praticos.py + polimento	compilar-para-pdf.py --tipo playbook

Critério de aceite F: `/produzir-obra-completa <tema>` com `gerar_playbook=true` entrega livro + `playbook` + derivados e o pacote `distribuicao/` lista o `playbook`.

6. Ordem de execução e dependências



Caminho crítico: A → B → D → F (as fases C e E podem correr em paralelo a D)

Sessão	Fases	Entrega verificável
1	A + B	20 passo_NN.json extraídos do livro AIDD v2, com relatório de lacunas
2	C + D + E	PLAYBOOK-aidd.pdf gerado pela esteira e aprovado no gate
3	F	/criar-playbook e /produzir-obra-completa operando ponta a ponta

7. Caminho enxuto (10 h) — se o objetivo for validar rápido

Corta o que não bloqueia a primeira entrega:

Corte	Economia	Consequência
Usar template.typ do livro com -V sem_cip=1 em vez de criar template_playbook.typ	-2 h	PDF sai com estilo de livro; card sem caixa de destaque
Adiar coletar_playbook() e reusar coletar()	-1 h	Playbook carrega CIP/ISBN desnecessários
validar-playbook.py só com R-PBK-1 a R-PBK-3	-1,5 h	Gate mais frouxo na v1

Recomendação: cortar apenas o template_playbook.typ na v1 (a diferença visual é cosmética e o template pode ser adicionado depois sem migração). Manter o gate completo — é ele que impede o playbook de virar um resumo do livro, que é o modo de falha mais provável desta etapa.

8. Riscos e mitigações

Risco	Prob.	Impacto	Mitigação
Capítulo sem §4/§5 quebra a extração	Média	Alto	<code>auditar-obra.py</code> já exige as 7 seções antes da Fase 3 — o <code>playbook</code> só roda em obra CONFORME; o extrator degrada para “card parcial” e reporta
Playbook vira resumo do livro (repete teoria)	Alta	Alto	R-PBK-0 no gate: re-prova se o card contiver texto das seções §1–§3 acima de um limiar de similaridade (reusa <code>jaccard/shingles</code> de <code>auditar-obra.py</code>)
Variação de acentuação nos cabeçalhos (Técnica vs Tecnica)	Média	Médio	Reusar <code>sem_acento()</code> de <code>auditar-obra.py</code> na comparação — já existe
Obras antigas (V3) sem <code>motivo_condutor</code>	Média	Baixo	Fallback: estágios nomeados pelas Partes; sem vocabulário <code>condutor</code> , sem falha
Grafo desatualizado (build em 1348aba, HEAD em 5d4a03b)	Alta	Baixo	Rodar <code>rebuild</code> do <code>.code-review-graph</code> antes da Fase B
Playbook de obra XG estourar contexto no <code>polimento</code>	Baixa	Médio	<code>Polimento</code> em lotes de 5 cards via <code>pool-capitulos.py --manifesto</code> (mecanismo já existe)

9. Impacto em tokens

Etapa	Natureza	Custo
Fatiamento	Script	0
Extração dos 20 cards	Script	0
Objetivo do Material + ligações de estágio	LLM	~3 k tokens
Polimento de lacunas apontadas pelo relatório	LLM	~1 k por card com lacuna
Ilustração de capa	Playwright/HTML	0 (sem API)
Compilação	Pandoc+Typst	0 (isento por CLAUDE.md §0.5)

Estimativa para um livro XG (20 capítulos): 5–12 k tokens de saída — cerca de **1/10 do custo de um e-book**, que reescreve a prosa inteira.

10. Definição de pronto (aceite final da etapa)

- ☐ `python scripts/fatiar-obra.py <slug> --playbook` cria a estrutura e grava derivados.json
- ☐ `python scripts/extraair-passos-praticos.py <slug>` produz 1 card por capítulo, sem lacuna crítica
- ☐ Capa gerada com badge de nível aprovado por `validar-capa-nivel.py`
- ☐ Ilustração da capa reflete o motivo condutor do livro-mãe
- ☐ `python scripts/validar-playbook.py <slug-pbk> --estrito` retorna 0
- ☐ `python compilar-para-pdf.py playbooks/<slug>--pbk --tipo playbook` gera o PDF
- ☐ `pytest tests/ -q verde` (incluindo as 2 novas suítes)
- ☐ `/criar-playbook` e `/produzir-obra-completa` documentados em CLAUDE.md
- ☐ `empacotar-distribuicao.py` inclui o playbook no pacote e no README
- ☐ Regressão: gerar um e-book e um artigo continua funcionando sem alteração

11. Prova de conceito já disponível

O arquivo `output/livros/ai-driven-development-do-zero-ao-deploy-v2/PLAYBOOK-aidd.md (+ .pdf)` foi produzido manualmente a partir do livro AIDD v2 e serve como **especificação viva** do formato-alvo: é o resultado que a etapa automatizada deve reproduzir sozinha. Use-o como caso de teste de referência da Fase B — o extrator está pronto quando gerar cards equivalentes aos daquele documento sem intervenção humana.